

Planning Fast and Slow: Guided Trajectory Synthesis via Conditional Flow Matching

Frank Tsu-Fang Lu

Master of Science in Computer Vision, Robotics and Machine
Learning
from the
University of Surrey



School of Computer Science and Electrical and Electronic
Engineering
Faculty of Engineering and Physical Sciences
University of Surrey
Guildford, Surrey, GU2 7XH, UK

September 2025

Supervised by: Dr. Simon Hadfield

DECLARATION OF ORIGINALITY

I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

MSc Dissertation Title: Guided Trajectory Synthesis via Flow Matching

Author Name: Frank Tsu-Fang Lu

Author Signature

Date: 2 September 2025

Supervisor's name: Simon Hadfield

WORD COUNT

Number of Pages:

Number of Words:

ABSTRACT

This should provide a summary of what highlights the reader is going to expect to read. The length should be short, 200-300 words. An abstract should include: The significance of the work, which may require a brief overview of the topic area. The contributions that this work is making to the topic area, the key results that are presented Achievements to date, what they were and how they are paving the path for future work.

Note that this is the first thing that the examiner is going to read possibly after taking a glance at the title. They need to be informed about what they are going to expect to find in this report.

CONTENTS

Declaration of Originality	ii
Word Count	iii
Abstract	iv
List of figures	vii
1 Introduction	1
1.1 Background and Context	2
1.2 Objectives	3
1.3 Achievements	4
1.4 Overview of Dissertation	4
2 Background Theory and Literature Review	6
2.1 Offline Reinforcement Learning	6
2.1.1 Policy Constraint Methods	6
2.1.2 Implicit Policy Constraint Methods	7
2.1.3 Model-based Offline RL Methods	8
2.1.4 Conservative Q Learning	8
2.2 Diffusion	9
2.3 Flow Matching	9
2.4 Planning with Diffusion	9
2.5 Other Relevant Works	9
2.6 Summary	10
3 Flow Matching Methodologies	11
3.1 First Section	11
3.1.1 First Subsection	11

3.1.1.1	First Subsubsection	11
3.1.2	Second Subsection	11
3.2	Second Section	11
3.3	Third Section	12
4	Dataset Preparation	13
4.1	Section	13
4.2	Section	13
5	Experimental Results	14
5.1	Architectural Selection	14
5.2	Modeling Joint Distribution	14
5.3	Testing Different Conditioning Signals	14
5.4	Hierarchical Planning	14
5.4.1	Global Planner	14
5.4.2	Local Planner	14
6	Conclusions	15
6.1	Evaluation	15
6.2	Future Work	15
	Bibliography	16
	Appendix	17

LIST OF FIGURES

3.1	Example figure and caption.	12
-----	-------------------------------------	----

1 INTRODUCTION

Imbuing intelligent agents with the capability for autonomous-decision-making in real-world environments is challenging. For decades, reinforcement learning has offered a robust mathematical framework for acquiring optimal behavioral skills through interactions within an environment to maximize cumulative rewards. The integration of deep neural networks as high-capacity function approximators has propelled RL even farther, leading to remarkable achievements across diverse domains, from mastering complex games to enabling basic robotic locomotion and manipulation.

However, the widespread success of modern machine learning, as seen in areas such as computer vision and natural language processing, fundamentally hinges on the availability of large and highly diverse datasets. In the conventional RL paradigm, the acquisition of such extensive data becomes prohibitively impractical for real-world applications. For example, autonomous driving would need to practice in numerous cities for every model update, which is costly, time-consuming, let alone being unsafe. This inherent limitation created a gulf of generalization between lab-bound RL successes and the complex, open-world settings where intelligent decision-making is most critically needed.

Offline Reinforcement Learning emerges as a pivotal paradigm to bridge this gap. Offline RL focuses on enabling the training of policies solely from previously collected datasets, without additional online interaction. This approach holds immense potential for transforming vast amounts of historical data into robust, generalizable decision-making engines, much like how supervised learning revolutionized pattern recognition. The implications for fields such as robotics, healthcare, and autonomous driving are profound, as it offers a viable pathway to deploy sophisticated AI systems where active data collection is either unfeasible or too costly. Beyond these, offline RL can revolutionize domains such as operations research, logistics, power plant control, inventory management, and finance, all of which have rich historical data where active exploration is difficult or expensive.

Finally, taking a page out of the rise of vision language models, the ability to leverage large and diverse datasets in an offline setting further enables the development of

foundational Vision-Language-Action models or generalist robot models. This means that pre-training on massive, heterogenous and potentially multi-modal datasets can lead to acquisition of broadly generalizable skills. These can then be efficiently fine-tuned for specific tasks in deployment. In a way, this paradigm allows bootstrapping on pre-trained visual language models to learn behaviors. Passive data can now be transformed into active intelligence by learned observations.

1.1 Background and Context

Despite its immense promise, offline RL is plagued by algorithmic challenges. At its core, offline RL is fundamentally about making and answering what Levine et al. describe as “counterfactual queries”. These are “what if” questions about what might happen if the agent were to carry out actions different from those in the dataset. In other words, for an agent to improve upon the behavior policy, it must be able to execute action sequences that diverge from the training data. This inherent need for counterfactual inference strains conventional machine learning tools, which assume that test data is drawn from the same distribution as training data. Consequently, making such counterfactual queries results in distributional shift. When a learned policy deviates from the training data, conventional RL algorithms can produce erroneous and excessively optimistic Q-value estimates, leading to brittle policies that perform poorly when deployed [1].

This instability is particularly exacerbated by high-dimensional function approximators such as Deep Neural Networks (DNNs). The errors can accumulate over long horizons, making stable learning notoriously difficult. Furthermore, real-world datasets, especially those from human demonstrations, inherently exhibit multimodal action distribution: there are often multiple valid ways to perform a task. Traditional methods like behavior cloning struggle to capture this multimodality, often averaging across modes and leading to indecisive or jerky behaviors that fail to generalize [2].

To address the challenge of multimodality, generative modeling, particularly with denoising diffusion models, has emerged as a powerful approach. By framing planning as a conditional sampling process, diffusion models can learn and represent complex, multimodal distributions directly from data. This is a critical capability for robotics, where a task may have multiple valid solutions, such as different grasps for an object or various

paths around an obstacle. By capturing the entire distribution of expert solutions, these models can enhance a robot’s versatility and ability to generalize [2]-[4].

However, the very properties that make diffusion models powerful for generation introduce fundamental challenges for their deployment in real-time robotic systems. These models are built upon a stochastic, iterative framework that imposes a significant computational burden, leading to high-latency inference incompatible with dynamic control loops. More importantly, their stochastic nature is fundamentally misaligned with the deterministic physical laws and hard safety constraints that govern robotic motion, calling for an alternative generative paradigms.

Therefore, this dissertation presents Flow Planner, a hierarchical framework built upon flow matching, a powerful alternative that directly addresses the core limitations of diffusion models. Flow matching learns a deterministic transformation governed by an Ordinary Differential Equation (ODE) rather than a Stochastic Differential Equation (SDE). This fundamental distinction makes it not merely an incremental improvement but a paradigm inherently aligned with the deterministic, low-latency requirements of physical robotic systems [5]-[7].

1.2 Objectives

The primary aim of this dissertation is to develop and validate a novel, deterministic framework for behavior cloning (BC) in offline reinforcement learning, leveraging the principles of flow matching. The specific objectives established to achieve this aim are as follows:

- To design and validate a systematic methodology for curating offline reinforcement learning datasets from actively trained agents.
- To design and implement "Flow Planner," a modular framework that utilizes flow matching for the generation of trajectories from offline data.
- To analyze the impact of various conditioning mechanisms on the precision and generalizability of the trajectories generated by the framework.
- To evaluate the performance of the "Flow Planner" framework, benchmarking its core attributes such as inference speed and policy effectiveness against diffusion models.

1.3 Achievements

This dissertation’s primary contribution is the development and validation of Flow Planner, a novel, modular framework for deterministic behavior cloning. The key achievements are:

- **A Data Curation Pipeline and Benchmark Datasets:** A flexible pipeline was designed and implemented for curating offline reinforcement learning datasets. This was demonstrated by producing three distinct datasets from PPO agents in Box2D environments, with the system designed for easy extension to other agents and environments.
- **A Modular Framework for Behavior Cloning:** The Flow Planner framework was developed to directly address the high latency and stochasticity that limit current diffusion-based BC approaches. Its modular architecture is a key feature, allowing for the scalable optimizations of its different components.
- **Analysis of the Flow Planner Framework:** This work provides a comprehensive analysis of the architectural choices for applying flow-matching models to BC. The findings shows the critical role of conditioning mechanisms and model architecture, offering practical guidance for future implementations.
- **Successful Proof-of-Concept:** The Flow Planner framework was successfully validated in the LunarLander continuous control environment. This proof-of-concept demonstrated the ability to generate effective policies with significantly lower inference latency than the baseline diffusion approach.

1.4 Overview of Dissertation

This dissertation is structured as follows:

- Chapter 2 provides the theoretical foundation for this work, first examining established methods for addressing distribution shift problems, then exploring contemporary solutions that leverage generative models.

- Chapter 3 details the methodology for data curation and introduces the experimental environments. This includes a technical discussion on the implementation of the data pipeline using Gymnasium wrappers and the Minari library for dataset management.
- Chapter 4 presents the core technical foundation of this dissertation. It details various approaches to dataset modeling with comprehensive mathematical formulations. The chapter ends by introducing "Flow Planner," the proposed hierarchical planning framework, and provides implementation specifics using Meta's flow matching library.
- Chapter 5 provides a comprehensive analysis and discussion of the empirical results. This includes an investigation of various conditioning mechanisms and a benchmark comparison of the framework's performance against baseline models.
- Chapter 6 concludes the dissertation by summarizing the key contributions, discussing the implications of the findings, and outlining several promising directions for future research.

2 BACKGROUND THEORY AND LITERATURE REVIEW

This chapter begins by examining the fundamental concepts in offline reinforcement learning and established methodologies for addressing distributional shift. Next, it explores diffusion models through the lens of variational bayesian framework, before unifying these concepts through the lens of continuous flows to introduce flow matching. The chapter concludes with an analysis of diffusion applications in trajectory synthesis and state-of-the-art approaches that integrate these models directly into reinforcement learning frameworks. Particular emphasis is placed on the mathematical formulations of diffusion and flow matching, as this constitutes the central focus of this dissertation and the subsequent methodologies.

2.1 Offline Reinforcement Learning

As mentioned in chapter 1, a significant hurdle in offline RL is distribution shift between the static dataset and the policy being learned. This results in standard off-policy RL methods to fail due to overestimation of Q-values, particularly when dealing with complex and multi-modal data distributions. When an agent’s policy deviates from the offline dataset, it queries the Q-function for actions not present in the training data. This leads to erroneous and optimistically high value estimates. The problem is especially severe when the policy is chosen to maximize the Q-value, thereby exploiting regions of the state-action space where the Q-function is highly inaccurate. As a result, the Q-function believes it will perform exceptionally well, while actual performance is often poor.

2.1.1 Policy Constraint Methods

One early class of methods that’s been explored extensively is policy constraint methods. These approaches aim to mitigate distributional shift by updating the policy not just by maximizing the Q-value, but by doing so *subject* to a constraint that the learned policy remains sufficiently close to the behavioral policy (collected data). This closeness is typically measured by a divergence metric, such as KL divergence or Maximum Mean Discrepancy (MMD).

To a degree, these methods can bound the distributional shift and the impact of out-of-distribution actions, potentially leading to more stable learning. Examples include **BEAR** (Bootstrapping Error Accumulation Reduction) by Kumar et al and **BRAC** (Behavior Regularized Actor Critic) by Wu et al [8], [9].

A significant challenge is their potential to be overly conservative, as a single constraint value ϵ might not work well across all states, especially if the behavior policy exhibits both highly random and highly deterministic actions. More critically, these methods often require **accurately estimating the behavior policy itself**, which can be extremely difficult, particularly when the data comes from diverse sources (e.g., humans or a mixture of different policies). Errors in this estimation can lead to an imperfect constraint and poor practical performance, making it a major bottleneck, especially during online fine-tuning.

2.1.2 Implicit Policy Constraint Methods

Building upon the insights into the limitations of explicit policy constraint methods, **implicit policy constraint methods**, such as **AWAC** (Advantage Weighted Actor Critic) by Nair et al. , emerged as an alternative that addresses the difficulty of behavior policy modeling. AWAC leverages a theoretical identity that shows the solution to a constrained optimization problem can be approximated using a **weighted maximum likelihood objective**. In this formulation, actions taken from the behavior policy are weighted by the exponentiated advantage function, effectively embedding the constraint implicitly [10]. More intuitively, AWAC approximates the best new policy by look at actions already in the dataset and give more weight (advantage) to the really good ones.

This approach **obviates the need for explicit modeling of the behavior policy**, only requiring samples from it, which are readily available in the offline dataset. AWAC has shown to be effective for online fine-tuning after offline initialization and demonstrates strong performance on complex tasks like dexterous manipulation with human demonstration data.

While the sources highlight its advantages, it is still a form of policy constraint. Its inherent conservativeness might limit the extent to which it can discover entirely novel, optimal behaviors far from the observed data.

2.1.3 Model-based Offline RL Methods

Taking a different algorithmic direction, **model-based offline RL methods**, exemplified by **MOPO** (Model-based Offline Policy Optimization) by Yu et al., propose to tackle the problem by learning a model of the environment’s dynamics. While vanilla model-based RL (like MBPO) already showed some promise in offline settings, MOPO specifically modifies these algorithms to handle distributional shift. MOPO achieves this by **punishing the policy for exploiting out-of-distribution states** during model rollouts. This is implemented by subtracting an **uncertainty penalty** from the reward function, which becomes larger for state-action tuples that are more out of distribution (i.e., where the learned model’s predictions are less certain) [11], [12].

MOPO hence has the ability to **generalize beyond the state and action support of the offline data**, enabling the learning of policies for tasks different from those for which the data was collected. It is more resilient to overestimation and overfitting than model-free methods and theoretically maximizes a lower bound of the policy’s true return, balancing exploration and risk. Empirically, MOPO outperforms existing model-free offline RL methods and vanilla model-based approaches, especially on tasks requiring out-of-distribution generalization.

Nevertheless, models can still be inaccurate in regions far from the training data, and in a purely offline setting, these errors cannot be corrected through further interaction. The effectiveness relies heavily on the quality of the uncertainty estimation and the tuning of the penalty coefficient. It may also struggle with datasets lacking sufficient action diversity, and there are limits to its generalization capabilities if the new task requires exploring states completely outside the available data support.

2.1.4 Conservative Q Learning

Finally, a distinct and highly effective approach is **Conservative Q-Learning (CQL)** by Kumar et al., which directly addresses the core problem of **Q-value overestimation**. Instead of constraining the policy, CQL augments the standard Bellman error objective with a novel **Q-value regularizer**. This regularizer works by finding and minimizing (pushing down) erroneously high Q-values for out-of-distribution actions, while a more

advanced version also compensates by pushing up Q-values for actions present in the dataset. This makes the learned Q-function inherently "pessimistic".

CQL is theoretically guaranteed to learn a **lower bound on the true Q-function**, effectively preventing the problematic overestimation. This direct approach leads to state-of-the-art performance across a wide range of tasks on standard benchmarks like D4RL and Atari. It significantly outperforms many previous offline RL methods, demonstrating the ability to "stitch" together parts of different trajectories to find more optimal paths, and achieving up to five times better results on challenging dexterous manipulation tasks. While conservative, the full CQL method only slightly underestimates Q-values, making it a robust and reliable algorithm.

The effectiveness of CQL depends on the appropriate selection of the regularization weight (α). Simpler variants of the algorithm without the compensatory term for in-distribution actions can lead to excessive underestimation. Despite its strong performance, ongoing research continues to explore further improvements.

In conclusion, policy constraint methods and implicit policy constraint methods both try to constrain the learned policy to remain close to the behavior policy. However, they do not aim to learn a pessimistic Q-function or ensure a lower bound on Q-values. They simply avoid the problematic evaluation of out-of-distribution action. On the other hand, MOPO and CQL are methods that embrace a conservative and "pessimistic" strategy in order to mitigate the risks associated with going out of distributions and value overestimation by either directly pushing down the Q-values or penalized rewards based on uncertainty. This ensures the learned policies are robust, accepting the trade-off that may lead to sub-optimal but reliable performance.

2.2 Diffusion

2.3 Flow Matching

2.4 Planning with Diffusion

2.5 Other Relevant Works

Note that references work like this [3] [2]

2.6 Summary

Your final section should include a summary of the chapter, which will summarise what has been covered and where there are gaps in knowledge which pave the way into the original contributions you make in the coming chapters.

3 FLOW MATCHING METHODOLOGIES

The technical body of the report consists of a number of chapters (just one here, but there will usually be more) of work you have carried out to date. Follow a logical structure in how you present your work. This will usually be the phases of the work you carried out which will link loosely to the objectives. Note that these chapters are not meant to be a log book, thus do not write them like a diary of work carried out but as a technical report specifying clearly what has been implemented and presented in a way that someone else could carry out the work based on what they have read.

Again you need to consult with your supervisor on the technical chapters you plan to write by providing a draft contents page. It should show clearly the sections in it and every chapter should have a clear paragraph introducing it and what to expect in it.

3.1 First Section

Structure your text into sections.

3.1.1 First Subsection

If necessary, also use subsections. Subsections are entered using the Heading 3 paragraph style (all these heading styles are self-numbering).

3.1.1.1 First Subsubsection

If you really need subsubsections.

3.1.2 Second Subsection

And, as required, more subsections.

3.2 Second Section

As an example of a figure, consider Figure 3.1 and note how it is referenced here by cross referencing to a caption method so that also you can form a list of figures at the start. You should be familiar with how to insert a caption and refer to it in Word from the training needs analysis form you were given. The caption should include title and essential

information about figure and figures must always be referred to in the text. Ensure you insert figures into appropriate points in the text so they are both near to the text that is referring to the figure but also ensure that they are arranged in a tidy fashion so they do not leave big gaps with no text.

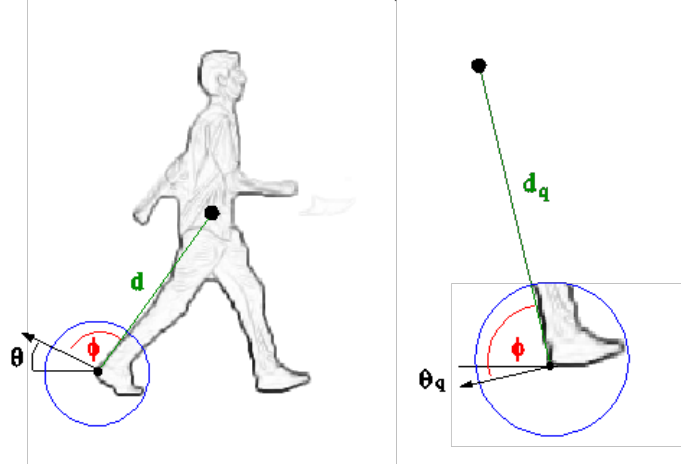


Figure 3.1: Example figure and caption.

Table 3.1: Make sure to set caption to large size.

Test	Test
More	Testing
More	Testing

3.3 Third Section

This sections hows how to do equations and to show that they number correctly as LaTeX can do with the chapter number and equation number to refer to equation in a piece of text:

$$y = x^2 \tag{3.1}$$

but note it is usually after putting in an equation that you may refer back to equation (3.1) later in the document or chapter.

4 DATASET PREPARATION

You need to work out how many technical chapters you will require, possibly more than two in this template. Normally your final chapter before the conclusion will include some clear results or proof of concepts from the end goals of your dissertation and it is important to document these well and show the value of what you have achieved. Frequently many dissertations fail to document this information correctly because it is rushed at the end so do ensure you allow time to write it properly.

4.1 Section

Text to insert

4.2 Section

Text to insert – but ensure the last section really does show the value of your work and where you have got to.

5 EXPERIMENTAL RESULTS

5.1 Architectural Selection

5.2 Modeling Joint Distribution

5.3 Testing Different Conditioning Signals

5.4 Hierarchical Planning

5.4.1 Global Planner

5.4.2 Local Planner

6 CONCLUSIONS

Your conclusion should summarise what you have carried out in this work to date, what you have achieved and what you are on course to achieve. Also it should explain what was documented in this dissertation. Note that someone may read this chapter without bothering to read the earlier chapters so there is no problem with repeating what you may have already written earlier. Indeed no new information should be in this part of the conclusion. It should reference back to what is already there.

6.1 Evaluation

Stand back and evaluate what you have achieved and how well you have met the objectives. Evaluate your achievements against your objectives in section 1.2. Demonstrate that you have tackled the project in a professional manner.

6.2 Future Work

Discuss what further work could be carried out from this work and relate it to what has been carried out.

BIBLIOGRAPHY

- [1] ESA. LEON's First Flights. [http://www.esa.int/Our_Activities/Space_Engineering_Technology/LEON_s_first_flights/\(print\)](http://www.esa.int/Our_Activities/Space_Engineering_Technology/LEON_s_first_flights/(print)), 2016. Accessed: 3-12-2016.
- [2] RTEMS C User Guide. RTEMS C User Guide. On-Line Applications Research Corporation, Feb. 2013.
- [3] Wall, J. V. , Shimmins, A. J. , and Bolton, J. G. . The Parkes 2700 MHz Survey (Ninth Part): Supplementary Catalogue for the Declination zone -45° to -65° . Australian Journal of Physics Astrophysical Supplement, 1975.
- [4] Wertz, J. R. and Larson, W. J. . Space Mission Analysis and Design. Space Technology Library. Springer Netherlands, 1999. ISBN 9780792359012.

APPENDIX

You may have one or more appendices containing detail, bulky or reference material that is relevant though supplementary to the main text: perhaps additional specifications, tables or diagrams that would distract the reader if placed in the main part of the dissertation. Make sure that you place appropriate cross-references in the main text to direct the reader to the relevant appendices. Again you should discuss with your supervisor what appendices are appropriate for this work.

Note that you should not include your program listings as an appendix or appendices. You should submit one copy of such bulky text as a separate item, perhaps on a disk.